

Chapitre 1

La compilation par l'exemple

Notes rédigées par Raphaël JONTEF

cours dispensé par M. Janin

Enseirb-Matmeca
filière informatique

3 septembre 2026

Table des matières

I	Analyse lexicale	2
II	Analyse syntaxique	2
III	Analyse sémantique	2
IV	Génération de code	3

Il existe trois couches principales dans un compilateur :

- l'analyse lexicale : elle consiste à transformer une suite de caractères en une suite de mots. Cette phase est réalisée par un analyseur lexical.
- l'analyse syntaxique : elle consiste à vérifier que la suite de mots obtenue par l'analyse lexicale respecte les règles de grammaire du langage. Cette phase est réalisée par un analyseur syntaxique.
- l'analyse sémantique : elle consiste à vérifier que le programme est sémantiquement correct, c'est-à-dire qu'il respecte les règles de validité des opérations et des identificateurs. Cette phase est réalisée par un analyseur sémantique.

I Analyse lexicale

L'analyse lexicale consiste en la conversion de l'une suite de caractères en une suite de mots.

Exemple 1.1

p	=	p0	+	v	*	t
ID	EQ	ID	PLUS	ID	TIMES	ID

Pour parvenir à cela, l'analyseur lexical va utiliser un ensemble de règles de reconnaissance de mots, appelées *expressions régulières*. Ces expressions régulières sont des motifs qui décrivent les séquences de caractères qui forment des mots valides dans le langage. On utilise des analyseurs lexicaux comme `flex` ou `lex` pour générer automatiquement le code de l'analyseur lexical à partir de ces expressions régulières.

II Analyse syntaxique

Pour analyser la syntaxe de la suite de mots obtenue par l'analyse lexicale, on utilise un analyseur syntaxique. L'analyse syntaxique consiste à vérifier si la suite de mots respecte les règles de grammaire du langage. Cette grammaire spécifie donc la syntaxe (priorités de calculs, associativité, etc.) du langage. L'analyse syntaxique est souvent réalisée à l'aide d'arbres syntaxiques abstraits (AST) qui représentent la structure hiérarchique de la suite de mots.

On utilise des analyseurs syntaxiques comme `bison` ou `yacc` pour générer automatiquement le code de l'analyseur syntaxique à partir de la grammaire du langage.

III Analyse sémantique

Après l'analyse syntaxique, on obtient une représentation structurée du programme, sous la forme d'un arbre syntaxique abstrait (AST). Cependant, cette représentation ne garantit pas que le programme est sémantiquement correct.

Exemple 1.2

Le programme peut multiplier un entier par un flottant, ce qui est une opération soit à invalider, soit à convertir implicitement. L'analyse sémantique va donc vérifier la validité des opérations et effectuer les conversions nécessaires.

Il y a donc au cours de cette phase de l'analyse sémantique, deux tâches principales :

- analyse de type : vérifier que les opérations sont valides pour les types d'opérandes donnés, et effectuer les conversions nécessaires si possible.
- analyse de noms : vérifier que les identificateurs utilisés dans le programme sont déclarés et accessibles dans le contexte actuel.

IV Génération de code